

Clear - Complete Integration Guide for Claude Code

Date: December 19, 2025

Task: Integrate Lovable design system into existing clear-app

STEP 0: CREATE BACKUP (DO THIS FIRST!)

```
bash

# Navigate to parent directory
cd ~

# Create timestamped backup of current clear-app
cp -r clear-app clear-app-backup-$(date +%Y%m%d-%H%M%S)

# Verify backup was created
ls -la | grep clear-app-backup

# Expected output: You should see a folder like "clear-app-backup-20251219-123045"
```

If anything goes wrong, you can restore with:

```
bash

cd ~
rm -rf clear-app
cp -r clear-app-backup-XXXXXXXX-XXXXXX clear-app
```

STEP 1: SET UP STAGING AREA

```
bash
```

```
cd ~/clear-app
```

```
# Create staging directory
```

```
mkdir -p lovable-staging
```

```
# Copy Lovable project contents to staging
```

```
cp -r /home/claude/* lovable-staging/
```

```
# Verify copy
```

```
ls -la lovable-staging/
```

Expected output: Should see all Lovable files (package.json, src/, public/, etc.)

STEP 2: UPDATE DESIGN SYSTEM FOUNDATION

2a. Replace Tailwind Config

```
bash
```

```
cd ~/clear-app
```

```
# Backup current tailwind config
```

```
cp tailwind.config.js tailwind.config.js.backup 2>/dev/null || cp tailwind.config.ts tailwind.config.ts.backup 2>/dev/null
```

```
# Copy Lovable's Tailwind config
```

```
cp lovable-staging/tailwind.config.ts ./
```

```
# Verify
```

```
cat tailwind.config.ts | grep "clear-purple"
```

Expected output: Should see the clear brand colors defined

2b. Replace index.css

```
bash
```

```
# Backup current CSS
cp src/index.css src/index.css.backup

# Copy Lovable's CSS
cp lovable-staging/src/index.css src/

# Verify
cat src/index.css | grep "clear-purple"
```

Expected output: Should see HSL color variables defined

2c. Copy Background Images

```
bash

# Copy assets folder
cp -r lovable-staging/src/assets src/

# Verify images exist
ls -la src/assets/
```

Expected output: Should see bg-mobile.png and bg-desktop.png

STEP 3: INSTALL DEPENDENCIES

```
bash
```

```
cd ~/clear-app
```

```
# Install new dependencies from Lovable
```

```
npm install @radix-ui/react-accordion @radix-ui/react-alert-dialog @radix-ui/react-aspect-ratio @radix-ui/react-avatar @radix-ui/react-checkbox @radix-ui/react-collapsible @radix-ui/react-context-menu @radix-ui/react-dialog @radix-ui/react-dropdown-menu @radix-ui/react-hover-card @radix-ui/react-label @radix-ui/react-menubar @radix-ui/react-navigation-menu @radix-ui/react-popover @radix-ui/react-progress @radix-ui/react-radio-group @radix-ui/react-scroll-area @radix-ui/react-select @radix-ui/react-separator @radix-ui/react-slider @radix-ui/react-slot @radix-ui/react-switch @radix-ui/react-tabs @radix-ui/react-toast @radix-ui/react-toggle @radix-ui/react-toggle-group @radix-ui/react-tooltip
```

```
npm install class-variance-authority clsx tailwind-merge
```

```
npm install tailwindcss-animate
```

```
# Install dev dependencies
```

```
npm install -D @types/node
```

```
# Verify installation
```

```
npm list | grep radix
```

Expected output: Should see all Radix UI packages installed

STEP 4: COPY SHADCN/UI COMPONENTS

```
bash
```

```
cd ~/clear-app
```

```
# Copy entire ui components folder
```

```
cp -r lovable-staging/src/components/ui src/components/
```

```
# Verify
```

```
ls -la src/components/ui/ | wc -l
```

Expected output: Should see ~40+ component files

STEP 5: COPY UTILITY FILES

```
bash
```

```
# Copy lib folder (utility functions)
```

```
cp -r lovable-staging/src/lib src/
```

```
# Copy hooks folder
```

```
cp -r lovable-staging/src/hooks src/
```

```
# Copy types folder
```

```
cp -r lovable-staging/src/types src/
```

```
# Copy components.json (shadcn config)
```

```
cp lovable-staging/components.json ./
```

```
# Verify
```

```
cat src/lib/utils.ts
```

Expected output: Should see the cn() utility function

STEP 6: COPY CUSTOM COMPONENTS

```
bash
```

```
cd ~/clear-app/src/components
```

```
# Copy Clear-specific components (not in ui/ folder)
```

```
cp ~/clear-app/lovable-staging/src/components/AnchorGrid.tsx ./
```

```
cp ~/clear-app/lovable-staging/src/components/ExerciseCard.tsx ./
```

```
cp ~/clear-app/lovable-staging/src/components/GenerateButton.tsx ./
```

```
cp ~/clear-app/lovable-staging/src/components/Header.tsx ./
```

```
cp ~/clear-app/lovable-staging/src/components/IntensitySlider.tsx ./
```

```
cp ~/clear-app/lovable-staging/src/components/LocationAccordion.tsx ./
```

```
cp ~/clear-app/lovable-staging/src/components/NavLink.tsx ./
```

```
cp ~/clear-app/lovable-staging/src/components/OptionalFields.tsx ./
```

```
cp ~/clear-app/lovable-staging/src/components/ReviewHeader.tsx ./
```

```
cp ~/clear-app/lovable-staging/src/components/StartWorkoutButton.tsx ./
```

```
cp ~/clear-app/lovable-staging/src/components/WorkoutOverview.tsx ./
```

```
cp ~/clear-app/lovable-staging/src/components/WorkoutSectionCard.tsx ./
```

```
# Copy workout subfolder
```

```
mkdir -p workout
```

```
cp -r ~/clear-app/lovable-staging/src/components/workout/* ./workout/
```

```
# Verify
```

```
ls -la
```

```
ls -la workout/
```

Expected output: Should see all custom components

STEP 7: COPY PAGE COMPONENTS

```
bash
```

```
cd ~/clear-app
```

```
# Create pages folder if it doesn't exist
```

```
mkdir -p src/pages
```

```
# Copy all page components
```

```
cp lovable-staging/src/pages/Index.tsx src/pages/
```

```
cp lovable-staging/src/pages/GenerationScreen.tsx src/pages/
```

```
cp lovable-staging/src/pages/ReviewScreen.tsx src/pages/
```

```
cp lovable-staging/src/pages/WorkoutScreen.tsx src/pages/
```

```
cp lovable-staging/src/pages/SummaryScreen.tsx src/pages/
```

```
cp lovable-staging/src/pages/NotFound.tsx src/pages/
```

```
# Verify
```

```
ls -la src/pages/
```

Expected output: Should see 6 page files

STEP 8: UPDATE APP.TSX AND MAIN.TSX

8a. Update main.tsx

```
bash
```

```
# Backup current main
```

```
cp src/main.tsx src/main.tsx.backup
```

```
# Copy Lovable's main.tsx
```

```
cp lovable-staging/src/main.tsx src/
```

```
# Verify
```

```
cat src/main.tsx
```

Expected output: Should see React Router setup

8b. Update App.tsx

```
bash
```

```
# Backup current App
cp src/App.tsx src/App.tsx.backup

# Copy Lovable's App.tsx
cp lovable-staging/src/App.tsx src/

# Verify
cat src/App.tsx
```

STEP 9: UPDATE INDEX.HTML

```
bash

cd ~/clear-app

# Backup current index.html
cp index.html index.html.backup

# Copy Lovable's index.html (includes Oxanium font)
cp lovable-staging/index.html ./

# Verify fonts are loaded
cat index.html | grep "Oxanium"
```

Expected output: Should see Oxanium font in the <head>

STEP 10: UPDATE PACKAGE.JSON (If Needed)

```
bash

# Check if React Router is installed
npm list react-router-dom

# If not installed, add it
npm install react-router-dom
```

STEP 11: TEST LOCALLY

```
bash  
  
cd ~/clear-app  
  
# Start dev server  
npm run dev
```

Expected output:

- Server starts on localhost:5173
- No errors in terminal
- Open browser to <http://localhost:5173>

What to check:

1. Background gradient is visible
 2. Fonts look correct (Rajdhani headers, Inter body)
 3. Generation screen loads with intensity slider and anchor grid
 4. Colors match Cyberpunk aesthetic
-

STEP 12: FIX COMMON ISSUES (If Needed)

Issue: TypeScript errors about missing types

```
bash  
  
npm install -D @types/react @types/react-dom
```

Issue: Module not found errors

```
bash  
  
# Clear node_modules and reinstall  
rm -rf node_modules package-lock.json  
npm install
```

Issue: Vite config errors

```
bash
```

```
# Copy Lovable's vite config
```

```
cp lovable-staging/vite.config.ts ./
```

STEP 13: COMMIT TO GIT

```
bash
```

```
cd ~/clear-app
```

```
# Check what changed
```

```
git status
```

```
# Stage all changes
```

```
git add .
```

```
# Commit with descriptive message
```

```
git commit -m "Integrate Lovable design system v2"
```

- Updated design system with semantic color mappings
- Added shadcn/ui component library
- Implemented all 3 core screens (Generation, Review, Workout)
- Fixed background gradient display
- Added Oxanium font for cyberpunk aesthetic
- All original color values preserved"

```
# Push to GitHub
```

```
git push origin main
```

STEP 14: VERIFY DEPLOYMENT

```
bash
```

```
# Vercel should auto-deploy on push to main
# Check deployment status at: https://vercel.com/dashboard

# Or manually trigger deploy:
vercel --prod
```

Check deployed site: <https://clear-app-ashen.vercel.app/>

ROLLBACK INSTRUCTIONS (If Something Goes Wrong)

```
bash

cd ~

# Delete broken version
rm -rf clear-app

# Restore from backup
cp -r clear-app-backup-XXXXXXXX-XXXXXX clear-app

# Reinstall dependencies
cd clear-app
npm install

# Test
npm run dev
```

SUCCESS CRITERIA

After integration, you should have:

- ☒ Background gradient visible (gold/blue)
- ☒ All 3 screens working (Generation, Review, Workout)
- ☒ Sharp corners (zero border radius)
- ☒ Glassmorphic cards with blur effects
- ☒ Orange glowing CTAs

-  Cyberpunk aesthetic throughout
 -  No console errors
 -  Deployed to Vercel
-

NOTES FOR CLAUDE CODE

- Execute steps in order
- After each step, verify the expected output
- If a step fails, STOP and report the error
- Test locally (Step 11) before committing
- Keep backup until user confirms everything works

Estimated time: 15-20 minutes for full integration